

GHOSTPRINT

IoT device identification via network-traffic fingerprinting

RAIoT Lab — Amity University Rajasthan · Innovation Day 2026



WHAT IT IS

GhostPrint identifies the IoT devices on a local network from how they communicate — packet sizes, inter-packet timing, packet rate and burstiness — not from what they transmit. A Random Forest classifies each device from a 12-second behavioural window and never reads a payload byte, so identification works identically on encrypted traffic. Two anomaly detectors flag rogue devices and impersonation (spoofing) attacks in real time.

HOW IT WORKS

1 · Simulate

3 ESP8266 profiles +
rogue & spoofer
boards, or the software
twin

2 · Capture

UDP metadata only:
size, timing, source IP

3 · Fingerprint

12 s rolling window → 4
behavioural features

4 · Classify

Random Forest: label +
confidence score

5 · Detect & show

rogue / spoofing alerts
→ live dashboard

THE BEHAVIOURAL FINGERPRINT

One 12-second window per device yields four features. The three demo profiles sit far apart in this space:

Device	Avg packet size	Inter-packet interval	Packets / min	Burstiness
SensorNode	72 B	5,000 ms	12	0.08
CameraStream	1,200 B	33 ms (in-burst)	55	0.24
SmartSwitch	32 B	1,800 ms	33	0.50

Source: GhostPrint device profiles (config.py) — the same profiles generate the training corpus.

ATTACK DETECTION

ROGUE DEVICE

An unknown device joins the network: its feature vector sits far from every known device profile (scale-aware z-distance), or the classifier is unconfident. Amber warning naming the source IP.

Alert in ≤ 5 s

SPOOFING ATTACK

An impostor takes over a known device's identity: a gated per-device z-score baseline spots the timing drift, and attack traffic never reshapes the reference. Red critical alert naming the victim.

Alert in ~10 s

VERIFIED RESULTS

100%

5-fold CV accuracy

25

automated tests

22

SRS requirements

≤ 5 s

rogue alert

~10 s

spoofing alert

Accuracy: 5-fold cross-validation over the 1,500-window training corpus. Latencies: scripted end-to-end runs (scripts/smoke_test.py replays the full attack scenario headlessly).

RUN THE DEMO

```
pip install -r requirements.txt
python scripts/run_demo.py
open http://localhost:5000
```

(no ESP8266 hardware required)

Click Rogue Device for an amber alert in ≤ 5 s; select SensorNode under Spoofing Attack for a red critical alert in ~10 s.
Stack: Python 3.10 · scikit-learn · Flask + Socket.IO · Chart.js · scapy · ESP8266 — thresholds in one config.py.